Data Structures - Detailed Answer Key (20 Marks)

Detailed Answer Key for Subjective Questions:

1. 1. Explain the difference between a singly linked list and a doubly linked list.
Answer: A singly linked list is a linear data structure where each node contains two fields: data and a pointer to the next node. It supports efficient memory utilization as memory is allocated dynamically. However, traversal is possible only in one direction. Insertion and deletion operations are straightforward, especially at the head or in the middle. But searching takes linear time as it involves traversing each node.

A doubly linked list, on the other hand, has an additional pointer that points to the previous node. This bidirectional nature allows both forward and backward traversal. This makes certain operations like deletion and insertion easier and more efficient, particularly when the node reference is known. However, it uses more memory due to the extra pointer and has slightly more complex management compared to singly linked lists. Doubly linked lists are commonly used in scenarios requiring frequent insertion and deletion, such as in browsers' back-and-forward functionality, whereas singly linked lists are often used in applications where memory is limited and insertions are mainly at one end.

2. 2. Describe the concept of dynamic memory allocation in data structures.
Answer: Dynamic memory allocation refers to the process of allocating memory at runtime rather than during the compilation phase. It provides flexibility, allowing programs to manage memory based on current requirements. Functions like malloc(), calloc(), realloc(), and free() in C/C++ are commonly used for this purpose.

When memory is allocated dynamically using malloc or calloc, it returns a pointer to the allocated memory block. Realloc can be used to resize the memory block if needed. After using the memory, it is essential to free the allocated memory using free() to avoid memory leaks. Unlike static memory allocation, which has a fixed size determined at compile time, dynamic allocation is efficient in managing memory usage and reducing wastage.

Applications such as managing large datasets, dynamic arrays, and linked lists heavily rely on dynamic memory allocation. However, improper memory management can lead to issues like memory leaks, fragmentation, and dangling pointers, so using it with caution is necessary.

3. 3. What is a priority queue? How is it implemented using a heap?

Answer: A priority queue is a special type of queue where each element has a priority. Elements with higher priority are served before elements with lower priority. If two elements have the same priority, they are typically processed in the order they were added (FIFO).

Priority queues are commonly implemented using heaps, specifically binary heaps. A binary heap maintains a complete binary tree structure, ensuring logarithmic time complexity for insertion and deletion operations. Min-heaps store the minimum value at the root, while max-heaps store the maximum value at the root. The most common operations in a priority queue include inserting an element, removing the highest-priority element, and peeking at the highest-priority element.

Applications of priority queues include task scheduling in operating systems, shortest path algorithms like Dijkstra's, and data compression using Huffman coding. Their efficient management of priorities makes them essential in time-critical applications.

4. 4. Explain the concept of binary search and its time complexity.

Answer: Binary search is a highly efficient search algorithm used on sorted arrays. The key principle is to divide the search space into two halves by comparing the target value to the middle element. If the target is equal to the middle element, the search ends. If the target is smaller, the algorithm continues searching in the left half. If larger, it searches the right half.

The process is repeated until the target is found or the search space is empty. The time complexity of binary search is O(log n) due to the continuous halving of the search space.

Binary search requires the input data to be sorted, making sorting algorithms like Merge Sort or Quick Sort useful beforehand. It is widely used in applications such as searching in large databases, finding elements in dictionaries, and locating words in a dictionary app.

Its logarithmic time complexity makes it one of the most effective algorithms for searching large datasets.

5. 5. Discuss the applications of graph data structures in real life.

Answer: Graphs are one of the most versatile data structures used to represent relationships between objects. A graph consists of nodes (vertices) connected by edges, which can be directed or undirected.

In social networks, graphs are used to model relationships where nodes represent users, and edges

represent friendships or follows. Platforms like Facebook and LinkedIn use graph algorithms to suggest connections and find mutual friends.

In navigation systems like Google Maps, graphs represent road networks. Nodes are intersections or places, and edges are roads connecting them. Algorithms like Dijkstra's or A find the shortest path between locations.*

Computer networks use graphs to represent routers and data pathways. Network routing algorithms optimize data packet transfer.

Biological systems also apply graph theory to model genetic networks and molecular interactions. Even in recommendation systems used by platforms like Netflix and Amazon, graphs analyze user preferences and suggest content based on similarities.

Overall, graphs are a powerful tool for solving real-world problems where relationships and connections are crucial.